

1 Write a Python program using lambda functions to sort a list of dictionaries.

Aim

To write a Python program that sorts a list of dictionaries using lambda functions as the key for sorting.

Algorithm

1. Create a list of dictionaries with sample data (e.g., students with and).
2. Use the built-in function.
3. Pass a lambda function as the argument to specify which dictionary field to sort by.
4. Display the sorted list.
5. Demonstrate sorting by different keys (e.g., by and by).

CODE

```
# Program to sort a list of dictionaries using lambda
```

```
students = [  
    {"name": "Yogin", "marks": 89},  
    {"name": "Priya", "marks": 95},  
    {"name": "Alex", "marks": 78},  
    {"name": "Sam", "marks": 88}  
]  
  
print("Original List of Dictionaries:")  
for s in students:  
    print(s)  
  
# Sort by name
```

```
sorted_by_name = sorted(students, key=lambda x: x["name"])
print("\nSorted by Name:")
for s in sorted_by_name:
    print(s)

# Sort by marks
sorted_by_marks = sorted(students, key=lambda x: x["marks"], reverse=True)
print("\nSorted by Marks (Descending):")
for s in sorted_by_marks:
    print(s)
```

OUTPUT

Original List of Dictionaries:

```
{'name': 'Yogin', 'marks': 89}
{'name': 'Priya', 'marks': 95}
{'name': 'Alex', 'marks': 78}
{'name': 'Sam', 'marks': 88}
```

Sorted by Name:

```
{'name': 'Alex', 'marks': 78}
{'name': 'Priya', 'marks': 95}
{'name': 'Sam', 'marks': 88}
{'name': 'Yogin', 'marks': 89}
```

Sorted by Marks (Descending):

```
{'name': 'Priya', 'marks': 95}
{'name': 'Yogin', 'marks': 89}
{'name': 'Sam', 'marks': 88}
```

```
{'name': 'Alex', 'marks': 78}
```

Conclusion

- The program demonstrates how to use lambda functions with `sorted()` to sort dictionaries.
- Sorting can be done by any key (, , etc.) and in ascending or descending order.
- This approach is concise, flexible, and widely used in Python for handling structured data.

2 Write a program to sort a list using a simple function.

Aim

To write a Python program that sorts a list of numbers in ascending order using a user-defined function.

Algorithm

1. Define a function .
2. Use a simple sorting technique (like bubble sort or selection sort).
3. Traverse the list and compare elements.
4. Swap elements if they are in the wrong order.
5. Return the sorted list.
6. Call the function with a sample list and display the result.

CODE

```
# Function to sort a list using bubble sort
```

```
def sort_list(lst):
```

```
    n = len(lst)
```

```
    for i in range(n):
```

```
        for j in range(0, n-i-1):
```

```
    if lst[j] > lst[j+1]:  
        # Swap elements  
        lst[j], lst[j+1] = lst[j+1], lst[j]  
    return lst
```

Sample list

```
numbers = [45, 12, 78, 34, 23, 89]
```

```
print("Original List:", numbers)
```

```
print("Sorted List:", sort_list(numbers))
```

OUTPUT

Original List: [45, 12, 78, 34, 23, 89]

Sorted List: [12, 23, 34, 45, 78, 89]

Conclusion

- The program demonstrates sorting a list using a simple user-defined function.
- Bubble sort is used here, but you can replace it with other algorithms like selection sort or insertion sort.
- This helps understand how sorting works internally instead of relying only on Python's built-in function.